

DailyDeck Developer Handbook

Step-by-Step Guide: Adding a New Activity (Section)

Schema Definitions • State Wiring • UI Controls • 16:9 Slide Preview • Slides Recipe Export

1. Overview of the Activity Lifecycle

In DailyDeck, an Activity (historically termed a Section) represents an instructional block that teachers can drag into their deck timeline, configure through the Inspector, preview on an exact 16:9 canvas, and compile into a Google Slides presentation. Adding an activity requires touching six key files in the codebase.

The Six Required Development Steps:

1 Define Metadata & Field Schema [[src/data/sections.js](#)]
Register the activity key, title, icon, colors, activity type (static/daily/weekly/mixed), and input fields.

2 Initialize Data Defaults [[src/ui/timeline.js](#)]
Provide default structures in `addTimelineItem()` for items added to the Deck Sequencer.

3 Configure Inspector Controls [[src/ui/inspector.js](#)]
Allow auto-generation from field schemas or attach specialized interactive UI inputs and speaker notes.

4 Build 16:9 Slide Preview Canvas [[src/ui/preview.js](#)]
Create an instant SVG/HTML slide simulation matching the template layout with day tabs and pagination.

5 Export to Presentation Recipe [[src/export/recipe.js](#)]
Translate timeline items into atomic slide commands with template tags `[noted]` and token replacements.

6 Register in Workspace Library [[src/main.js](#)]
Expose the activity in the Library panel for Literacy, Numeracy, or Starter Decks.

2. Activity Types (Data Modeling)

- **static:** Executes once per deck or has uniform data that does not vary across days (e.g. Intro Slide, Day Divider).
- **daily:** Contains separate data for each day of the week (Monday through Friday). Stored in `item.data[day][fieldId]`.
- **weekly:** Uniform configuration shared across all days of the deck. Stored in `item.data[fieldId]`.
- **mixed:** Combines deck-wide global settings (weeklyFields) with day-specific custom data (dailyFields).

3. Step 1: Define Metadata (src/data/sections.js)

Every activity must have an entry in SECTION_DEFS. This object serves as the single source of truth for UI icons, descriptions, activity types, and form input schemas.

src/data/sections.js — Activity Definition

```
// In src/data/sections.js -> export const SECTION_DEFS = { ... }
'sectNumberBonds': {
  title: 'Number Bonds',
  shortDesc: 'Practise part-part-whole number facts',
  icon: 'fa-diagram-project', // FontAwesome icon class
  color: 'text-amber-500',    // Icon text color class
  bg: 'bg-amber-100',        // Icon background class
  type: 'daily',              // 'static' | 'daily' | 'weekly' | 'mixed'
  hasTitleSlideOptions: true, // Enables "Include section title slide" toggle
  description: 'Students find missing parts of a number bond (part-part-whole).',
  fields: [
    { id: 'targetSum', label: 'Target Whole', type: 'select', options: ['5', '10', '20'], default:
'10' },
    { id: 'knownPart', label: 'Known Part', type: 'text', default: '7', placeholder: 'e.g. 7' },
    { id: 'missingPart', label: 'Missing Part (Answer)', type: 'text', default: '3', placeholder:
'e.g. 3' }
  ]
}
```

4. Step 2: Timeline Defaults (src/ui/timeline.js)

When a teacher adds the activity, addTimelineItem(defKey) instantiates the item. Standard fields defined in def.fields are populated automatically. For specialized arrays or custom defaults, add a hook:

src/ui/timeline.js — Data Initialization Hook

```
// In src/ui/timeline.js -> addTimelineItem(defKey, index, skipSelectAndRender)
if (defKey === 'sectNumberBonds') {
  DAYS.forEach(day => {
    if (!newItem.data[day]) newItem.data[day] = {};
    newItem.data[day].targetSum = '10';
    newItem.data[day].knownPart = day === 'Monday' ? '7' : '6';
    newItem.data[day].missingPart = day === 'Monday' ? '3' : '4';
  });
}
```

5. Step 3: Inspector Controls & Speaker Notes

If def.fields contains standard text/select/checkbox inputs, inspector.js auto-generates the editing interface. For activities with speaker notes or custom UI, add hooks in inspector.js:

src/ui/inspector.js — Speaker Notes Hook

```
// In src/ui/inspector.js -> getSpeakerNotesText(itemOrKey, data)
case 'sectNumberBonds':
  return `Prompt: What number pairs with ${dayData.knownPart || '7'} to make ${dayData.targetSum ||
'10'}?`;
```

6. Step 4: Slide Preview Canvas (src/ui/preview.js)

The slide preview renders an instantaneous 16:9 representation of what the student will see on screen. Add your case to `generateSlidePreviewHtml()` and create a custom renderer function.

src/ui/preview.js — 16:9 Slide Canvas Renderer

```
// In src/ui/preview.js -> generateSlidePreviewHtml(...)
switch (defKey) {
  // ...
  case 'sectNumberBonds':
    return renderSectNumberBondsPreview(dayData, item.customTitle);
}

// Preview Renderer Function
function renderSectNumberBondsPreview(dayData, customTitle) {
  const targetSum = dayData.targetSum || '10';
  const knownPart = dayData.knownPart || '7';
  const missingPart = dayData.missingPart || '?';

  return `
    <div class="w-full h-full flex flex-col items-center justify-between p-4 select-none bg-white
    relative">
      <div class="w-full flex items-center justify-between border-b border-slate-100 pb-1">
        <span class="text-xs font-bold text-slate-800 edu-font">${customTitle || 'Number Bonds'}</
        span>
        <span class="text-[10px] font-bold text-amber-600 bg-amber-50 px-2 py-0.5 rounded-full
        border border-amber-200">Bonds to ${targetSum}</span>
      </div>
      <!-- Diagram Body -->
      <div class="flex-1 flex flex-col items-center justify-center my-auto">
        <div class="w-14 h-14 rounded-full border-2 border-amber-500 bg-amber-50 flex items-center
        justify-center text-xl font-bold text-slate-900">
          ${targetSum}
        </div>
        <div class="w-24 h-4 flex justify-between px-3">
          <div class="w-0.5 h-full bg-slate-300 transform -rotate-45"></div>
          <div class="w-0.5 h-full bg-slate-300 transform rotate-45"></div>
        </div>
        <div class="flex items-center gap-8">
          <div class="w-12 h-12 rounded-full border-2 border-slate-300 bg-slate-50 flex items-
          center justify-center text-lg font-bold text-slate-700">
            ${knownPart}
          </div>
          <div class="w-12 h-12 rounded-full border-2 border-dashed border-amber-500 bg-amber-50
          flex items-center justify-center text-lg font-bold text-amber-700">
            ${missingPart}
          </div>
        </div>
      </div>
      <div class="text-[10px] text-slate-500 edu-font text-center">What is the missing part?</div>
    </div>
  `;
}
```

7. Step 5: Export to Presentation Recipe (src/export/recipe.js)

When the user clicks "Assemble Deck", buildPresentationRecipe() compiles timeline items into slide instructions dispatched to the Google Apps Script backend (Code.gs). Each slide requires a notelId, replacements map, and optional injectNotes.

src/export/recipe.js — Slide Assembly Logic

```
// In src/export/recipe.js -> inside the daily iteration loop:
if (item.defKey === 'sectNumberBonds') {
  const dayData = (data && data[day]) || {};
  const targetSum = dayData.targetSum || '10';
  const knownPart = dayData.knownPart || '7';
  const missingPart = dayData.missingPart || '3';

  // Slide 1: Question Slide
  recipe.slides.push({
    noteId: "[numberBond_question]",
    replacements: {
      "{{targetSum}}": targetSum.toString(),
      "{{knownPart}}": knownPart.toString(),
      "{{missingPart}}": "?"
    },
    injectNotes: `Target Whole: ${targetSum}\nKnown: ${knownPart}\nAsk students for the missing part.`
  });

  // Slide 2: Reveal / Solution Slide
  recipe.slides.push({
    noteId: "[numberBond_answer]",
    replacements: {
      "{{targetSum}}": targetSum.toString(),
      "{{knownPart}}": knownPart.toString(),
      "{{missingPart}}": missingPart.toString()
    },
    injectNotes: `Solution: ${knownPart} + ${missingPart} = ${targetSum}`
  });
}
```

8. Step 6: Master Template Alignment (Google Slides)

The cloud backend (Code.gs) matches the recipe.slides notelId against the Speaker Notes of the master template presentation.

- **1. Add Template Slide:** In Google Slides, create the slide layout matching your activity.
- **2. Set Note Tag:** Open the Speaker Notes panel of that slide and enter the exact tag: [numberBond_question].
- **3. Insert Tokens:** In the text boxes on that slide, type placeholder tokens: {{targetSum}}, {{knownPart}}, {{missingPart}}.

9. Step 7: Library Registration (src/main.js)

Expose the new activity in the Library panel. In src/main.js -> renderLibrary():

- **Numeracy Workspace:** Add the key to numeracySectionKeys: const numeracySectionKeys = ['mainIntro', 'sectNumberBonds'];
- **Literacy Workspace:** All items in SECTION_DEFS are shown by default, or filter out numeracy-specific keys if appropriate.
- **Starter Decks:** Optionally append the key to starter decks in src/data/starters.js under sections: [...].

10. Developer Quality Checklist

- **Syntax Check:** Run node -c on all modified JavaScript files to catch any syntax issues.
- **Drag & Drop:** Verify that dragging from Library to Deck Sequencer instantiates a timeline item.

- **State Mutation:** Verify that typing into Inspector inputs updates data and persists across reload.
 - **Preview Fidelity:** Ensure the 16:9 canvas scales properly and renders user inputs accurately.
 - **Recipe Inspection:** Click Assemble Deck -> Toggle JSON recipe to verify notId and {{token}} mappings.
-

